SUPER DEV BUILDERS · DOKUMENTASI PENGGUNA

Panduan Lengkap SDB v0.7.1 — Cara Pakai yang Baik dan Benar

Semua skenario pemakaian: hari pertama sebagai admin, membuat dan memantau job, menjalankan worker, alur uang (hold → tagih → lepas), invoice bulanan, pemulihan setelah kegagalan, batas laju, metrik, isolasi antar tenant, migrasi & cadangan, sampai daftar lengkap kode galat.

v0.7.1 | 401 tes | 21 bagian | tanpa dependensi runtime

Dokumen ini ditulis untuk tiga pembaca sekaligus: **pemilik platform** (admin), **pengguna** (tenant yang membeli kapasitas), dan **operator pekerja** (worker). Setiap skenario memuat langkah, contoh perintah nyata, hasil yang diharapkan, dan apa yang harus dilakukan bila hasilnya berbeda.

> Versi yang didokumentasikan: **v0.7.1** (401 tes). Semua contoh memakai `http://127.0.0.1:8088` dan `data/control.db`; ganti bila port/nama berkas berbeda.

Daftar Isi

DAFTAR ISI

Daftar Isi

1. Apa itu SDB
2. Peta arsitektur
3. Prasyarat dan pemasangan
4. Konfigurasi dan variabel lingkungan
5. Menjalankan control plane
6. Skenario A — Hari pertama sebagai admin
   - A1. Membuat tenant
   - A2. Mengubah status tenant
   - A3. Menambah kredit (topup)
   - A4. Melihat pendapatan agregat
7. Skenario B — Tenant membuat job
   - B1. Membuat job
   - B2. Melihat status job
   - B3. Melampirkan artefak
8. Skenario C — Worker mengerjakan job
   - C1. Menjalankan worker
   - C2. Protokol yang dijalankan worker
   - C3. Klaim ganda dicegah
9. Skenario D — Uang: hold, tagih, lepas, refund
10. Skenario E — Bulan pertama: usage, invoice, pendapatan
    - E1. Melihat pemakaian
    - E2. Menerbitkan invoice (admin)

E3. Rekonsiliasi bulanan (rutinitas yang disarankan)

11. Skenario F — Pembatalan, kegagalan, dan pemulihan
12. Skenario G — Nyaman tapi aman: idempotensi & retry
13. Skenario H — Batas laju dan kuota
14. Skenario I — Metrik dan pemantauan
15. Skenario J — Dashboard Workspace (agar tidak salah pakai)
16. Skenario K — Isolasi antar tenant
17. Skenario L — Migrasi, cadangan, pemulihan

    Migrasi

    Cadangan

    Pemulihan

18. Semua kemungkinan galat
19. Daftar periksa keamanan sebelum online
20. Pertanyaan yang sering diajukan
21. Lampiran: contoh skrip klien Python

---

1. Apa itu SDB

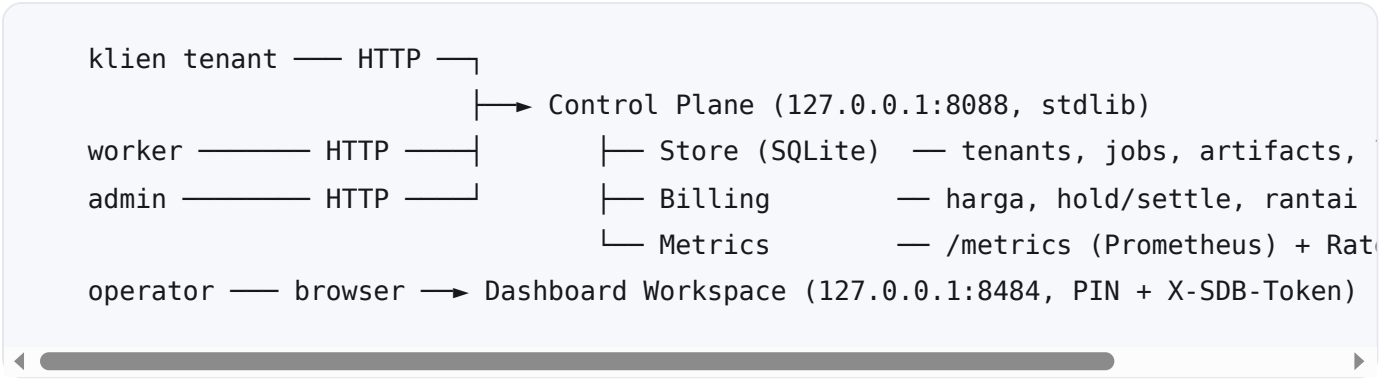
SDB (Super Dev Builders) adalah **platform pekerjaan berbasis kapasitas**: pelanggan mengirim pekerjaan (job), worker menjalankannya, dan pemakaian token ditagih dari saldo prabayar pelanggan. Yang dijual bukan "langganan software" tetapi **eksekusi yang terukur**: setiap job punya anggaran token dan waktu, setiap rupiah/token tercatat di buku besar ber-rantai hash.

Tiga janji yang dijaga oleh kode, bukan oleh janji pemasaran:

1. **Uang tidak bisa hilang diam-diam.** Setiap perubahan saldo adalah baris buku besar; baris baru menyimpan hash baris sebelumnya, jadi penghapusan/perubahan baris lama terdeteksi.
2. **Satu tenant tidak bisa membaca atau menagih tenant lain.** Kunci API terikat ke satu tenant, dan alamat artefak wajib berada di ruang tenant itu sendiri.
3. **Pekerjaan yang tidak selesai tidak ditagih penuh.** Saldo ditahan (hold) saat job dibuat, lalu ditagih sesuai pemakaian nyata; sisa hold dilepas.

---

2. Peta arsitektur



Komponen	Berkas	Tanggung jawab
Control plane	`sdb_platform/control_plane/app.py`	16 rute HTTP, autentikasi 3 peran (tenant/worker/admin)
Store	`sdb_platform/control_plane/store.py`	skema, transaksi, kuota, status job, artefak
Billing	`sdb_platform/control_plane/billing.py`	harga per plan, hold/settle/refund, invoice, verifikasi rantai
Metrics	`sdb_platform/control_plane/metrics.py`	penghitung Prometheus + token bucket per tenant
Worker	`sdb_platform/runtime/worker.py`	klaim job ber-lease, heartbeat, eksekusi, lapor hasil
Dashboard	`dashboard/server.py`, `dashboard/ws_api.py`	Workspace lokal :8484 (bukan bagian penagihan)

Rute control plane:

Metode	Rute	Peran
GET	`/healthz`	publik (denyut)
GET	`/metrics`	admin
POST	`/v1/tenants`	admin (buat tenant + kunci)
POST	`/v1/admin/tenants/<id>/status`	admin (aktif / suspend / hapus)
POST	`/v1/credits/topup`	admin
POST	`/v1/admin/invoices`	admin
GET	`/v1/admin/revenue`	admin
POST	`/v1/jobs` · GET `/v1/jobs/<id>`	tenant
POST	`/v1/jobs/<id>/cancel` · `/artifacts`	tenant
GET	`/v1/usage` · `/v1/billing` · `/v1/ledger` · `/v1/invoices`	tenant
POST	`/v1/worker/claim`	worker
POST	`/v1/jobs/<id>/heartbeat` · `/complete`	worker

Penting: dashboard adalah alat kerja operator (ide → diskusi → draft), **bukan** antrean job SDB. Job yang ditagih selalu melalui control plane :8088.

---

3. Prasyarat dan pemasangan

Kebutuhan: Linux, Python **3.10+**, `make`, `git`. Tidak ada dependensi runtime pihak ketiga untuk control plane, worker, dan metrik (seluruhnya stdlib).

```
cd ~/super-dev-builders
python3 -V                      # harus >= 3.10
make doctor                     # pemeriksaan lingkungan
make test                       # seluruh tes harus PASS
make check                      # lint + tes + drift + hygiene + pack
```

Bila memakai paket rilis:

```
tar xzf sdb-0.7.0.tar.gz && cd sdb-0.7.0
python3 -m venv .venv && . .venv/bin/activate && pip install -e .
```

Hasil yang diharapkan: `make check` berakhir tanpa galat dan baris terakhir memuat `PASS`.

Bila gagal: jalankan `make test` sendiri untuk melihat nama tes yang merah. Jangan lanjut ke produksi sebelum hijau — kode yang tidak lulus tesnya tidak boleh melayani pelanggan.

---

4. Konfigurasi dan variabel lingkungan

Variabel	Wajib	Arti	Default
`SDB_ADMIN_KEY`	ya (server)	kunci admin ≥ 24 karakter	—
`SDB_WORKER_KEY`	untuk worker	kunci worker ≥ 24 karakter; tanpa ini endpoint worker → 503	—
`SDB_RATE_PER_MIN`	tidak	jatah laju default per tenant	`600`
`SDB_ALLOW_ECHO_EXECUTOR`	hanya uji	izinkan executor `echo` (menagih token fiktif)	tidak aktif

```
export SDB_ADMIN_KEY=$(python3 -c 'import secrets;print(secrets.token_urlsafe(32))')
export SDB_WORKER_KEY=$(python3 -c 'import secrets;print(secrets.token_urlsafe(32))'
```

Aturan keras:

- Jangan menyimpan kunci di berkas yang masuk git. Pakai `.env` (di-`.gitignore`) atau `systemd` `ExecStart` dengan `EnvironmentFile=` ber-mode `600`.

- Kunci yang pernah tampil di layar/log/dokumen dianggap **bocor** dan harus diganti (buat tenant/kunci baru, cabut yang lama).
- Jangan cetak kunci di laporan, tiket, atau chat.

---

5. Menjalankan control plane

```
make control-plane                      # memakai data/control.db, port 8088
# atau eksplisit:
python3 -m sdb_platform.control_plane.app --db data/control.db --host 127.0.0.1 --po
```

Pemeriksaan denyut:

```
curl -s http://127.0.0.1:8088/healthz
# {"status": "ok"}
```

Untuk produksi: jalankan di belakang reverse proxy ber-TLS, dengarkan hanya `127.0.0.1` atau soket internal, dan pasang unit `systemd` agar hidup lagi setelah reboot. Contoh unit:

```
[Unit]
Description=SDB control plane
After=network.target

[Service]
WorkingDirectory=/home/anashadi/super-dev-builders
EnvironmentFile=/home/anashadi/.config/sdb/control.env
ExecStart=/home/anashadi/super-dev-builders/.venv/bin/python -m sdb_platform.control_
Restart=on-failure

[Install]
WantedBy=default.target
```

---

6. Skenario A — Hari pertama sebagai admin

A1. Membuat tenant

```
curl -s -X POST http://127.0.0.1:8088/v1/tenants \
  -H "X-Admin-Key: $SDB_ADMIN_KEY" -H "Content-Type: application/json" \
  -d '{"plan":"pro","name":"Toko Maju","isolation":"pool"}'
```

Jawaban (dipotong):

```
{"tenant": {"tenant_id": "01J...", "plan": "pro", "status": "active",
            "jobs_per_month": ..., "tokens_per_month": ..., "storage_gb": ...},
 "api_key": "sdb_..."}
```

`api_key` **hanya muncul sekali di respons ini.** Simpan ke pengelola rahasia; jangan di chat/git. Kredit uji otomatis: starter 5 µUSD, pro 25 µUSD, team 100 µUSD, enterprise 0.

Plan dan harganya (µUSD per 1 juta token; 1 USD = 1.000.000 µUSD):

Plan	Harga / 1 juta token	Kredit uji
starter	2,00 USD	5 µUSD
pro	1,60 USD	25 µUSD
team	1,20 USD	100 µUSD
enterprise	1,00 USD	0

Batasan kuota bulanan default ditentukan plan; dapat ditimpa saat membuat tenant:

```
-d '{"plan":"pro","limits":{"jobs_per_month":500,"tokens_per_month":50000000,
     "storage_gb":20,"rate_per_min":1200}}'
```

`rate_per_min` (1..100000) mengatur jatah laju tenant itu; tanpa nilai → `SDB_RATE_PER_MIN`.

A2. Mengubah status tenant

```
curl -s -X POST http://127.0.0.1:8088/v1/admin/tenants/01J.../status \
  -H "X-Admin-Key: $SDB_ADMIN_KEY" -H "Content-Type: application/json" \
  -d '{"status":"suspended"}'
```

- `suspended` → job baru ditolak, **worker tidak lagi mengklaim** job tenant itu (biaya berhenti).
- `deleted` → antrean ditutup dan kredit yang ditahan dilepas.

A3. Menambah kredit (topup)

```
curl -s -X POST http://127.0.0.1:8088/v1/credits/topup -H "X-Admin-Key: $SDB_ADMIN_K
  -H "Content-Type: application/json" \
  -d '{"tenant_id":"01J...","amount_microusd":50000000,"memo":"topup-transfer-2026-1
```

`50.000.000 µUSD` = 50 USD. `memo` wajib dan berguna saat merekonsiliasi dengan mutasi bank.

A4. Melihat pendapatan agregat

```
curl -s http://127.0.0.1:8088/v1/admin/revenue -H "X-Admin-Key: $SDB_ADMIN_KEY"
```

Isinya total tagihan (`charge`), topup, refund, dan saldo agregat seluruh tenant — bahan laporan bulanan.

---

7. Skenario B — Tenant membuat job

B1. Membuat job

```
curl -s -X POST http://127.0.0.1:8088/v1/jobs \
  -H "X-API-Key: $SDB_TENANT_KEY" -H "Content-Type: application/json" \
  -d '{
    "type": "recon",
    "pack": {"id": "sdb-recon", "version": "1.0.0"},
    "target_ref": "artifact://01J.../targets/example.com.txt",
    "budget": {"tokens": 5000, "seconds": 600},
    "idempotency_key": "recon-2026-10-29-0001"
  }'
```

Jawaban: HTTP `201` dengan `{"job": {...}, "replayed": false}`.

Aturan bidang:

Bidang	Wajib	Aturan
`type`	ya	teks pendek, mis. `recon`, `review`, `security`
`pack`	ya	objek berisi `id` dan `version`
`target_ref`	ya	alamat target di ruang tenant (`artifact://<tenant_id>/…` atau `s3://<bucket>/<tenant_id>/…`)
`budget.tokens`	ya	1.000 … 2.000.000
`budget.seconds`	ya	60 … 14.400 (4 jam)
`idempotency_key`	ya	8 … 128 karakter, unik per tenant

Efek uang saat dibuat: saldo sebesar **anggaran token** ditahan (hold) dari kredit tenant. Bila kredit kurang → `402 insufficient_credit`; bila kuota bulanan habis → `402 quota_*_exceeded`.

B2. Melihat status job

```
curl -s http://127.0.0.1:8088/v1/jobs/01J... -H "X-API-Key: $SDB_TENANT_KEY"
```

Status yang mungkin: `queued`, `running`, `succeeded`, `failed`, `blocked`, `budget_exceeded`, `cancelled`.

B3. Melampirkan artefak

```
curl -s -X POST http://127.0.0.1:8088/v1/jobs/01J.../artifacts \
  -H "X-API-Key: $SDB_TENANT_KEY" -H "Content-Type: application/json" \
  -d '{"kind":"report","sha256":"<64 hex>","size":18432,
       "redaction_status":"redacted","retention_until":"2026-12-31",
       "uri":"artifact://01J.../jobs/01J.../report.md","media_type":"text/markdown"}
```

- `uri` **wajib** memuat `<tenant_id>` sendiri; `..` ditolak.
- `sha256` 64 heksadesimal huruf kecil; `size` positif.
- `retention_until` tanggal ISO di masa depan (disimpan sebagai UTC).

---

8. Skenario C — Worker mengerjakan job

C1. Menjalankan worker

```
export SDB_WORKER_KEY=...      # >= 24 karakter, sama dengan kunci yang dipakai serv
python3 -m sdb_platform.runtime.worker --db data/control.db \
  --worker-id worker-1 --executor echo --once
```

- `--executor` **wajib** (pilihan saat ini: `echo`). `echo` hanya untuk uji: butuh `SDB_ALLOW_ECHO_EXECUTOR=1`, dan **menagih token** atas pekerjaan yang tidak nyata — jangan dipakai pada tenant berbayar.
- Tanpa `--once`, worker berputar terus dengan jeda `--poll` (default 2 detik).
- `--lease-seconds` default 120. Job panjang mendapat heartbeat otomatis (interval = lease/3), jadi job sampai 4 jam tidak kehilangan lease.

C2. Protokol yang dijalankan worker

```
POST /v1/worker/claim  {"worker_id":"worker-1","lease_seconds":120}   -> job | null
   kerjakan …
POST /v1/jobs/<id>/heartbeat {"worker_id":"worker-1","lease_seconds":120}   (berkala
POST /v1/jobs/<id>/complete  {"worker_id":"worker-1","status":"succeeded",
                              "tokens_used":1500,"gate_verdict":{...},"error":null}
```

- `tokens_used` **harus** ≤ anggaran; bila lebih, status akhir otomatis `budget_exceeded` dan yang ditagih hanya sebesar anggaran.
- `status` penutup yang sah: `succeeded`, `failed`, `blocked`, `budget_exceeded`, `cancelled`. `queued` / `running` **ditolak** (`400 invalid_status`) — inilah yang dulu membuat job bisa dikerjakan ulang gratis.
- `gate_verdict` bila ada harus objek; `error` berupa teks.
- Bila worker mati di tengah jalan, lease habis → job kembali `queued` dan bisa diklaim worker lain. Bila job dibatalkan tenant dan worker tidak pernah kembali, reaper menutupnya sebagai `cancelled`.

C3. Klaim ganda dicegah

`/complete` hanya diterima dari **worker pemegang lease**. Worker lambat yang mencoba menutup job yang sudah diambil worker lain mendapat `409 lease_lost`.

---

9. Skenario D — Uang: hold, tagih, lepas, refund

Alur satu job:

```
create  → hold = harga(budget.tokens)          (saldo: kredit - hold)
finish  → charge = harga(tokens_used)          (hold dilepas penuh, kredit berkurang
        → release = hold - charge  (bila ada sisa)
```

Contoh angka nyata (plan pro, 1,6 µUSD per 1.000 token):

Langkah	Peristiwa	Saldo kredit	Ditahan
awal	topup 50.000.000	50.000.000	0
create	budget 5.000 token = 8.000	50.000.000	8.000
finish	tokens_used 1.518 → 2.429	49.997.571	0

Pembulatan **ke atas**, jadi tidak ada token gratis karena pembulatan.

Kasus khusus yang sudah ditangani kode:

Kejadian	Perilaku
Job dibatalkan setelah kerja berjalan	ditagih sesuai `tokens_used` yang dilaporkan
Job dibatalkan sebelum kerja	hold dilepas penuh, tagihan 0
`tokens_used` > anggaran	ditagih sebesar anggaran, status `budget_exceeded`
Worker hilang setelah klaim	lease habis → job kembali antre; tidak ada tagihan ganda
Refund oleh admin	baris `refund` dengan `memo` wajib; atomik

Cek buku besar tenant:

```
curl -s "http://127.0.0.1:8088/v1/ledger?limit=50" -H "X-API-Key: $SDB_TENANT_KEY"
curl -s http://127.0.0.1:8088/v1/billing -H "X-API-Key: $SDB_TENANT_KEY"
```

Verifikasi rantai (deteksi perubahan baris lama):

```
python3 -c "
from sdb_platform.control_plane.store import Store
s = Store('data/control.db')
print(s.billing.verify_chain('01J...'))"
# {'ok': True, 'entries': 42, 'balance': 49997571, 'held': 0}
```

---

10. Skenario E — Bulan pertama: usage, invoice, pendapatan

E1. Melihat pemakaian

```
curl -s http://127.0.0.1:8088/v1/usage -H "X-API-Key: $SDB_TENANT_KEY"
```

Berisi pemakaian job/token/storage bulan berjalan dibanding kuota plan.

E2. Menerbitkan invoice (admin)

```
curl -s -X POST http://127.0.0.1:8088/v1/admin/invoices -H "X-Admin-Key: $SDB_ADMIN_
  -H "Content-Type: application/json" \
  -d '{"tenant_id":"01J...","period_start":"2026-10-01","period_end":"2026-10-31"}'
```

Aturan yang ditegakkan:

- `period_start` < `period_end`, format ISO-8601; periode **wajib sudah tertutup** (tidak boleh menagih periode yang belum selesai).
- **Tidak boleh tumpang-tindih** dengan invoice lain tenant yang sama → `409 invoice_period_overlap`. (Dulu dua invoice bertumpuk menagih biaya yang sama dua kali.)
- Invoice diterbitkan admin, bukan tenant — mencegah tenant "memesan" periode aneh yang membuat invoice resmi bulanan tertolak.

Tenant melihat daftarnya: `GET /v1/invoices`.

E3. Rekonsiliasi bulanan (rutinitas yang disarankan)

1. `GET /v1/admin/revenue` → total tagihan, topup, refund.
2. Bandingkan dengan mutasi bank/penyedia pembayaran untuk kolom topup.
3. Untuk tiap tenant besar: `verify_chain` harus `ok: true`.
4. Simpan salinan `data/control.db` (lihat §17) sebelum dan sesudah penutupan bulan.

11. Skenario F — Pembatalan, kegagalan, dan pemulihan

Situasi	Yang harus dilakukan	Hasil
Tenant ingin membatalkan	`POST /v1/jobs/<id>/cancel`	bila belum dikerjakan: hold dilepas, status `cancelled`; bila sudah berjalan: ditagih pemakaian
Ingin menutup job dari sisi tenant	**tidak bisa** (dulu bisa — itulah cacatnya) → `410 endpoint_removed`	pakai `/cancel`, atau tunggu worker
Worker mati saat job berjalan	tidak ada aksi manual	lease habis → `queued` → worker lain klaim
Job macet `queued` setelah batal	tidak ada aksi manual	reaper menutup `cancelled` + lepas hold
Hasil pekerjaan salah, ingin ulang	buat job baru dengan `idempotency_key` **baru**	job lama tetap tercatat (jejak audit)
Worker melaporkan gagal	kirim `status:"failed"` + `error`	ditagih pemakaian nyata (bukan anggaran penuh)
Gate menolak hasil	`status:"blocked"` + `gate_verdict`	ditagih pemakaian; keputusan tercatat

Contoh pembatalan:

```
curl -s -X POST http://127.0.0.1:8088/v1/jobs/01J.../cancel -H "X-API-Key: $SDB_TENA
```

12. Skenario G — Nyaman tapi aman: idempotensi & retry

Jaringan gagal setelah server memproses permintaan? Klien **wajib** mengulang dengan `idempotency_key` yang sama:

- Key sama + isi sama → `200` dengan `"replayed": true` dan job yang sama (tidak ada hold baru).
- Key sama + isi **berbeda** → `409 idempotency_conflict` (dulu diam-diam mengembalikan job lain).

Pola klien yang benar:

```
key = f"recon-{task_id}"          # stabil untuk satu niat, bukan acak per percobaan
for percobaan in range(5):
    try:
        r = post("/v1/jobs", {...}, key=key)
        break
    except (TimeoutError, HTTPError) as e:
        if isinstance(e, HTTPError) and e.code == 409:
            raise                                  # isi berubah: perbaiki kode,
        time.sleep(2 ** percobaan)                 # 2,4,8,16 detik
```

Aturan retry:

- Retry hanya untuk galat **sementara**: timeout, 429 (hormati `Retry-After`), 5xx.
- Jangan retry 400/401/402/409/410 — itu keputusan, bukan gangguan.

---

13. Skenario H — Batas laju dan kuota

Tiga pagar berbeda, jangan tertukar:

Pagar	Kapan aktif	Jawaban	Tindakan klien
Laju per tenant	terlalu banyak permintaan/menit	`429 rate_limited` + `Retry-After`	tunggu `Retry-After`, turunkan laju
Kuota kuota bulanan	job/token/storage bulan ini habis	`402 quota_jobs_exceeded` dll.	upgrade plan / tunggu bulan berikut
Kredit	saldo < anggaran job	`402 insufficient_credit`	topup

Jatah default 600 permintaan/menit per tenant; burst 2 detik pertama lebih longgar (kapasitas 2×laju). Atur per tenant saat pembuatan, atau ubah bawaan server dengan `SDB_RATE_PER_MIN`.

Ukur kebutuhan sebelum menaikkan: satu klien sehat jarang melewati 60 permintaan/menit. Bila sering kena 429, biasanya ada loop tanpa jeda — perbaiki klien dulu.

---

14. Skenario I — Metrik dan pemantauan

```
curl -s http://127.0.0.1:8088/metrics -H "X-Admin-Key: $SDB_ADMIN_KEY" | head -20
```

Keluaran (format Prometheus):

```
# TYPE sdb_http_requests_total counter
sdb_http_requests_total{method="GET",route="/v1/usage",status="200"} 12
sdb_http_requests_total{method="POST",route="/v1/jobs",status="201"} 4
# TYPE sdb_jobs_created_total counter
sdb_jobs_created_total{type="recon"} 4
# TYPE sdb_jobs_completed_total counter
sdb_jobs_completed_total{status="succeeded"} 3
# TYPE sdb_tokens_used_total counter
sdb_tokens_used_total 1518
# TYPE sdb_uptime_seconds gauge
sdb_uptime_seconds 812.4
```

`prometheus.yml`:

```
scrape_configs:
  - job_name: sdb
    scheme: http
    authorization:
      type: Bearer            # bila memakai proxy yang menambahkan header; atau pak
    static_configs:
      - targets: ["127.0.0.1:8088"]
    metrics_path: /metrics
```

Karena `/metrics` memerlukan header `X-Admin-Key`, cara paling aman adalah menyisipkannya di reverse proxy internal (jangan menaruh kunci admin di berkas konfigurasi Prometheus yang terbaca banyak orang).

Ambang peringatan yang disarankan:

Metrik	Peringatan	Artinya
`sdb_rate_limited_total` naik terus	> 5% permintaan	klien looping atau jatah terlalu kecil
`sdb_jobs_completed_total{status="failed"}`	> 20% dalam 1 jam	executor/pekerjaan bermasalah
`sdb_http_requests_total{status="500"}`	> 0	bug; periksa stderr server
`sdb_uptime_seconds` turun	restart tak terduga	periksa `systemctl`/OOM

Catatan jujur: penghitung disimpan di memori proses — hilang saat restart dan tidak dijumlah antar instance.

---

15. Skenario J — Dashboard Workspace (agar tidak salah pakai)

Dashboard `:8484` adalah **kantor kerja operator**: menulis ide, berdiskusi dengan agen, membuat draft intake Gate #0, lalu menyerahkan tahap build/test/scan/deploy ke terminal. Ia **bukan** tempat job penagihan berjalan.

```
systemctl --user status sdb-command-deck     # layanan
python3 dashboard/server.py --show-token     # PIN untuk perangkat lain
python3 dashboard/server.py --health-check   # probe kesehatan (dashboard/state/heal
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8484/     # 200 = UI terla
```

Aturan pakai:

- Buka dari PC server (127.0.0.1) untuk melihat PIN; PIN hanya diberikan ke klien lokal.
- Setiap panggilan `/api/ws/*` wajib header `X-SDB-Token`.
- Lebih dari 30 kegagalan token per IP dalam 60 detik → `429` (sejak v0.7.1).
- Data di `~/sdb-workspace/` (projects.json, chats/, stages/) — **bukan** di git.
- Tahap 07–16 hanya menghasilkan `*.brief.md`; eksekusi tetap manual dengan bukti segar.

Kesalahan umum: mengira "Approve" di dashboard berarti pekerjaan sudah dijalankan. Yang terjadi adalah draft diterima; eksekusi tetap di terminal.

16. Skenario K — Isolasi antar tenant

Uji cepat bahwa isolasi benar-benar berlaku (lakukan di lingkungan uji, bukan produksi):

```
# 1) tenant B mencoba membaca job tenant A
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8088/v1/jobs/<job_A> -H "X
# harapan: 404 job_not_found (bukan 403 yang membocorkan keberadaan)

# 2) tenant B mencoba menaruh artefak di ruang A
curl -s -X POST http://127.0.0.1:8088/v1/jobs/<job_B>/artifacts -H "X-API-Key: $KEY_
  -H "Content-Type: application/json" \
  -d '{"kind":"report","sha256":"'$(printf 'a%.0s' {1..64})'","size":10,
       "redaction_status":"redacted","retention_until":"2026-12-31",
       "uri":"artifact://<tenant_A>/curian.txt","media_type":"text/plain"}'
# harapan: 400 invalid_uri
```

Isolasi bukan hanya soal kunci: `uri` artefak, kepemilikan lease, dan kepemilikan job semuanya diperiksa di dalam transaksi, bukan di lapisan tampilan.

17. Skenario L — Migrasi, cadangan, pemulihan

Migrasi

Membuka DB lama dengan kode baru melakukan migrasi otomatis dan **tidak menulis ulang baris lama**:

- v0.4.0 → v0.5.0: kolom-kolom billing awal.
- v0.5.0 → v0.6.0: `ledger.hv` (versi hash), indeks `idx_jobs_status`, `idx_charges_tenant`.
- v0.6.0 → v0.7.1: `tenants.rate_per_min`.

Prosedur aman:

```
cp data/control.db data/control.db.bak-$(date +%Y%m%d-%H%M)
python3 -c "from sdb_platform.control_plane.store import Store; Store('data/control.
python3 -c "
from sdb_platform.control_plane.store import Store
s=Store('data/control.db')
for t in s.db.execute('SELECT tenant_id FROM tenants'):
    r=s.billing.verify_chain(t['tenant_id'])
    print(t['tenant_id'], r['ok'], r['entries'], r['balance'])"
```

Cadangan

```
sqlite3 data/control.db ".backup 'backup-$(date +%F).db'"     # konsisten walau serv
```

Simpan juga salinan `control.db` **sebelum** setiap rilis baru; DB adalah sumber kebenaran uang.

Pemulihan

1. Hentikan server.
2. Ganti `data/control.db` dengan cadangan.
3. Jalankan server, cek `/healthz`, lalu `verify_chain` semua tenant.
4. Cocokkan total kredit dengan laporan terakhir.

Bila `verify_chain` gagal: **jangan** menambal hash. Pulihkan dari cadangan; kegagalan rantai berarti ada baris yang diubah di luar aplikasi.

---

18. Semua kemungkinan galat

HTTP	`code`	Arti	Tindakan
400	`invalid_json`	body bukan JSON sah	perbaiki body
400	`invalid_budget`	tokens/seconds di luar batas	1000..2.000.000 token, 60..14.400 detik
400	`invalid_idempotency_key`	panjang < 8 atau > 128	pakai kunci stabil 8–128 karakter
400	`invalid_uri`	`uri` keluar ruang tenant / `..`	pakai `<tenant_id>` sendiri
400	`invalid_status`	status penutup tidak sah	pakai salah satu status penutup
400	`invalid_gate_verdict`	`gate_verdict` bukan objek	kirim objek atau `null`
400	`invalid_lease` / `lease_seconds`	lease di luar 10..3600	10..3600
400	`invalid_worker`	`worker_id` kosong	kirim `worker_id`
400	`invalid_redaction` / `invalid_retention`	metadata artefak cacat	ISO-8601 masa depan, status redaksi sah
400	`weak_key`	kunci < 24 karakter	buat kunci acak ≥ 24
400	`invalid_limits`	batas plan bukan int positif	perbaiki nilai
413	`body_too_large`	body > 256 KB	kirim lebih kecil
401	`unauthorized`	kunci salah/kosong	periksa header
402	`insufficient_credit`	saldo < anggaran	topup
402	`quota_jobs_exceeded` / `quota_tokens_exceeded` / `quota_storage_exceeded`	kuota bulanan habis	upgrade/tunggu bulan baru

HTTP	`code`	Arti	Tindakan
404	`job_not_found`	job tidak ada **atau** milik tenant lain	periksa id
404	`tenant_not_found`	tenant tidak ada	periksa id
404	`invoice_not_found`	invoice tidak ada	periksa id
409	`idempotency_conflict`	kunci sama, isi beda	pakai kunci baru untuk niat baru
409	`key_exists`	kunci API sudah dipakai	buat kunci lain
409	`lease_lost`	bukan pemegang lease	hentikan; job sudah diambil worker lain
409	`lease_owner`	lease milik worker lain	sama seperti di atas
409	`invoice_period_overlap`	periode bertumpuk	pakai periode lain
410	`endpoint_removed`	`/finish` dihapus	pakai `/cancel` (tenant) atau `/complete` (worker)
400	`invalid_period`	periode invoice tidak sah atau belum berakhir	pakai tanggal ISO, periode yang sudah lewat
429	`rate_limited`	jatah laju habis	tunggu `Retry-After`
500	`internal_error`	bug	periksa stderr server; laporkan dengan waktu & rute
503	`worker_disabled`	`SDB_WORKER_KEY` tidak diset	set di server

Prinsip: `404` dipakai untuk "tidak ada **atau** bukan milikmu" agar keberadaan job tenant lain tidak bocor.

---

19. Daftar periksa keamanan sebelum online

- [ ] Control plane **tidak** mendengarkan `0.0.0.0` tanpa TLS; pakai reverse proxy + sertifikat.
- [ ] `SDB_ADMIN_KEY`, `SDB_WORKER_KEY`, kunci tenant disimpan di berkas `600` atau pengelola rahasia.
- [ ] `/metrics` tidak terbuka ke internet (butuh header admin).
- [ ] Dashboard hanya di LAN/loopback; PIN tidak dibagikan lewat chat.
- [ ] `.env` tidak masuk git (`git check-ignore .env` harus mengonfirmasi).
- [ ] Cadangan `control.db` harian + `verify_chain` mingguan.
- [ ] Log tidak memuat kunci/header/body (server hanya mencatat metode + rute saat 500).
- [ ] Rotasi kunci bila ada keraguan kebocoran; cabut kunci lama (`api_keys` dihapus, tenant tetap).

---

20. Pertanyaan yang sering diajukan

Apakah satu job bisa ditagih dua kali? Tidak. Hold dibuat sekali di transaksi pembuatan; penutupan job memakai transaksi yang sama untuk melepas hold dan menagih pemakaian. Penutupan ganda ditolak.

Bisakah tenant menghapus jejak tagihan? Tidak. Buku besar ber-rantai hash; baris diubah → `verify_chain` gagal. Baris lama tidak pernah ditulis ulang oleh migrasi.

Bagaimana kalau worker crash di tengah job? Lease habis (default 120 detik) → job kembali `queued`. Tidak ada tagihan ganda karena penagihan hanya terjadi saat penutupan yang sah.

Kenapa tenant tidak bisa menutup job sendiri? Karena itu membuat pekerjaan gratis: tenant bisa "menutup" job yang sedang dikerjakan worker. Sekarang tenant hanya bisa membatalkan, dan pembatalan setelah kerja berjalan tetap ditagih.

Apakah metrik bisa dipercaya setelah restart? Tidak untuk total jangka panjang — penghitung di memori. Untuk angka permanen, pakai buku besar/invoice.

Berapa lama artefak disimpan? Sesuai `retention_until` yang dicatat saat pendaftaran artefak; penghapusan fisik mengikuti kebijakan penyimpanan yang Anda tetapkan (belum otomatis di v0.7.1).

---

21. Lampiran: contoh skrip klien Python

```
"""Klien minimal SDB: buat job, tunggu selesai, cetak tagihan. Hanya stdlib."""
import json, time, urllib.error, urllib.request

BASE = "http://127.0.0.1:8088"
KEY = "kunci-tenant-anda-yang-panjang"

def call(method, path, body=None, tries=4):
    data = json.dumps(body).encode() if body is not None else None
    for n in range(tries):
        req = urllib.request.Request(BASE + path, data=data, method=method,
                                     headers={"X-API-Key": KEY, "Content-Type": "app
        try:
            with urllib.request.urlopen(req, timeout=30) as r:
                return json.loads(r.read() or b"{}")
        except urllib.error.HTTPError as e:
            payload = json.loads(e.read() or b"{}")
            code = payload.get("error", {}).get("code", "")
            if e.code == 429 and n < tries - 1:
                time.sleep(int(e.headers.get("Retry-After", "2")))
                continue
            raise SystemExit(f"{e.code} {code}: {payload}")
        except TimeoutError:
            if n == tries - 1:
                raise
            time.sleep(2 ** n)

job = call("POST", "/v1/jobs", {
    "type": "recon", "pack": {"id": "sdb-recon", "version": "1.0.0"},
    "target_ref": "artifact://<tenant_id>/targets/contoh.txt",
    "budget": {"tokens": 5000, "seconds": 600},
    "idempotency_key": "contoh-2026-10-29-01"})
print("job:", job["job"]["job_id"], "replayed:", job["replayed"])

for _ in range(120):                       # tunggu maksimal ~10 menit
    st = call("GET", f"/v1/jobs/{job['job']['job_id']}")["job"]
    if st["status"] in ("succeeded", "failed", "blocked", "budget_exceeded", "cancel
        break
    time.sleep(5)
```

```
print("status:", st["status"], "token:", st.get("tokens_used"))
print("billing:", call("GET", "/v1/billing"))
```

Dokumen ini adalah bagian dari rilis v0.7.1. Bila perilaku sistem berbeda dari yang tertulis di sini, yang benar adalah kode dan tesnya — laporkan perbedaannya agar dokumen diperbaiki.

Dokumentasi ini menyertai rilis SDB v0.7.1. Bila perilaku sistem berbeda dari yang tertulis, yang berlaku adalah kode dan tesnya — laporkan perbedaannya agar dokumen diperbaiki.